

DOCUMENTATION TECHNIQUE

TerriSchool35

Prédiction des Effectifs Scolaires

Rennes Métropole

Grégory CRESPIN

Projet P13 — Master Data Science / IA / ML

Table des matières

Table des matières	2
Introduction et mode d'emploi de ce document	4
Comment lire ce document	4
1. Découverte et recherche des données (Data Discovery)	5
1.1 Fiches détaillées des fichiers sources (dossier DATA/)	5
1. communes_rennes_metropole.csv	5
2. donnee_statistique_commune_rennes_metropole.csv	5
3. fr-en-ecoles-effectifs-nb_classes.csv	5
4. mutation_immobiliere_dvf_rm.csv	6
5. population-par-sexe-age-et-nationalite-par-commune-2014.csv	6
6. permis_de_contruire_global_rennes_metropole_historique.csv	6
7. referentiel-batiment-et-ses-donnees-descriptives-sur-rennes-metropole.csv	7
8. nombre-et-type-de-logement-a-la-parcelle-cadastrale-sur-le-territoire-de-rennes.csv	7
9. Ficdep22.csv (hors base PostgreSQL)	7
1.2 Traçabilité et cycle de vie de la donnée	7
2. Ingestion et stockage (ETL et base PostgreSQL)	9
2.1 Pipeline ETL (load_data.py)	9
Architecture et logique d'exécution	9
Diagramme du flux de l'ETL	9
Exemple simplifié : chargement des écoles	10
2.2 Schéma relationnel PostgreSQL	10
Diagramme simplifié des relations entre tables	11
Dictionnaire des tables et volumétries réelles	12
2.3 Vue SQL v_effectifs_commune_annee	12
3. Analyse exploratoire et R&D (notebooks et démarche intellectuelle)	13
3.1 Démarche et enchaînement	14
Étape 1 — Audit qualité et corrélations (notebook 00_eda_qualite_donnees)	14
Étape 2 — Exploration des effectifs scolaires (notebook 01_eda_effectifs)	14
Étape 3 — Feature engineering et encodage (notebook 02_feature_engineering)	15
Étape 4 — Optimisation d'hyperparamètres (notebook 03_hyperparameter_tuning)	15
Étape 5 — Évaluation et comparaison (notebook 04_ml_evaluation)	16
Étape 6 — Interprétation par SHAP (notebook 05_ml_shap)	17
Étape 7 — Validation spatiale et fuite de données (notebook 06_validation_spatiale)	18
Étape 8 — Dynamique temporelle et Prophet (notebook 07_series_chronologiques)	18
4. Industrialisation Python et suivi MLflow	19
4.1 Script d'entraînement (train.py)	19
4.2 Script d'inférence (predict.py)	19
Fonction d'impact immobilier	19

4.3 Intégration de MLflow	19
5. Exposition REST (API FastAPI).....	21
5.1 Architecture de l'API.....	21
5.2 Spécification des endpoints.....	21
GET /health	21
GET /communes/{code_insee}/effectifs.....	21
POST /predictions/logement.....	21
POST /predictions/batch	22
6. Interface utilisateur (tableau de bord Streamlit)	23
6.1 Architecture multi-pages.....	23
7. Intégration continue (CI)	24
Étapes de validation du pipeline	24
8. Déploiement recette et production (CD et infrastructure)	25
8.1 Fichier docker-compose.yml	25
8.2 Déploiement en recette (REC)	25
8.3 Déploiement en production (PROD)	25
9. Supervision et monitoring	26
9.1 Journalisation de l'ETL	26
9.2 Supervision système	26
10. Glossaire complet	27

Introduction et mode d'emploi de ce document

TerriSchool35 est un projet de **Data Science** (science des données) qui prédit combien d'élèves seront scolarisés dans chacune des 43 communes de Rennes Métropole, année après année. L'objectif concret est d'aider les collectivités à anticiper l'ouverture ou la fermeture de classes, le recrutement d'enseignants, et le dimensionnement des futures écoles — notamment quand un nouveau programme immobilier est construit.

Le projet part de fichiers de données brutes (population, permis de construire, effectifs scolaires...), les nettoie et les stocke dans une base de données, entraîne des modèles de **Machine Learning** (apprentissage automatique) sur ces données, puis expose ces modèles via une API et un tableau de bord visuel pour que des utilisateurs non-techniciens puissent obtenir des prédictions.

Comment lire ce document

Ce document suit le cycle de vie réel d'un projet de Data Science, dans l'ordre chronologique où les choses se sont produites :

- **Section 1** — quelles données ont été trouvées et à quoi servent-elles ?
- **Section 2** — comment ces données sont-elles nettoyées et stockées dans une base ?
- **Section 3** — comment l'équipe a-t-elle exploré les données et construit les modèles prédictifs, étape par étape ?
- **Section 4** — comment ce travail de recherche a-t-il été transformé en code de production réutilisable ?
- **Section 5** — comment un autre logiciel peut-il demander une prédiction (l'API) ?
- **Section 6** — à quoi ressemble l'interface que voit un utilisateur final ?
- **Section 7 et 8** — comment le code est-il testé et déployé automatiquement ?
- **Section 9** — comment vérifie-t-on que tout fonctionne correctement une fois en production ?
- **Section 10** — un glossaire regroupe toutes les définitions techniques du document.

Astuce de lecture : chaque terme en **gras** la première fois qu'il apparaît est expliqué dans le paragraphe qui suit, ou renvoyé au glossaire (section 10) s'il revient souvent.

1. Découverte et recherche des données (Data Discovery)

Avant d'écrire la moindre ligne de code de Machine Learning, il faut d'abord savoir **quelles données existent**, où les trouver, et si elles sont fiables. Cette étape s'appelle le **Data Discovery**. Le projet s'appuie sur 9 fichiers de données bruts au format CSV (tableur texte), fournis principalement par Rennes Métropole, l'Insee (Institut national de la statistique) et l'Éducation Nationale.

1.1 Fiches détaillées des fichiers sources (dossier DATA/)

Ces fichiers sont stockés localement dans le dossier DATA/ du projet. Pour chacun, la fiche ci-dessous indique d'où vient le fichier, à quoi il sert, sa taille approximative, son format, et sa fiabilité.

1. communes_rennes_metropole.csv

- **Provenance** : Open Data Rennes Métropole (portail de données ouvertes de la métropole).
- **Rôle** : référentiel administratif des communes qui composent Rennes Métropole — la liste officielle de base.
- **Volume** : environ 4 Ko (43 lignes, une par commune de l'EPCI — l'établissement public de coopération intercommunale que forme la métropole).
- **Format** : CSV avec séparateur point-virgule (;).
- **Fréquence de mise à jour** : annuelle, lors d'évolutions administratives.
- **Colonnes principales** : code_insee (identifiant unique officiel de la commune), nom_commune, population, superficie_km2, densite_hab_km2.
- **Qualité** : excellente. C'est la table pivot : toutes les autres tables du système s'y rattachent via le code_insee.

2. donnee_statistique_commune_rennes_metropole.csv

- **Provenance** : Insee / Rennes Métropole.
- **Rôle** : données statistiques historiques et projections de population par commune.
- **Volume** : environ 3 Mo (43 communes).
- **Format** : CSV, séparateur point-virgule.
- **Fréquence** : annuelle.
- **Colonnes principales** : code_insee, pop_2020, pop_2021, pop_estim_2023, pop_proj_2030, pop_proj_2040, cat_0_14_2020 (pourcentage de la population âgée de 0 à 14 ans), natalite_2022 (taux de natalité pour 1 000 habitants), log_collectif, log_individuel, plh_nb_log_a_const (logements à construire prévus par le Programme Local de l'Habitat, un document qui planifie la construction de logements).
- **Qualité** : très bonne — c'est la source majeure des indicateurs socio-démographiques du projet.

3. fr-en-ecoles-effectifs-nb_classes.csv

- **Provenance** : Ministère de l'Éducation Nationale.
- **Rôle** : effectifs d'élèves et nombre de classes par école, pour chaque rentrée scolaire — c'est la donnée que le projet cherche justement à prédire pour les années futures.

- **Volume** : environ 158 Mo (fichier national, toutes communes de France confondues).
- **Format** : CSV, séparateur point-virgule.
- **Fréquence** : annuelle.
- **Colonnes principales** : Rentrée scolaire, Commune, Numéro de l'école, Dénomination principale, Nombre total d'élèves, Nombre d'élèves en pré-élémentaire (maternelle), Nombre d'élèves en élémentaire, REP, REP+ (indicateurs d'éducation prioritaire).
- **Qualité** : moyenne. Le fichier est national et volumineux : il faut filtrer rigoureusement sur le code départemental "35" (Ille-et-Vilaine) puis sur les seules communes de Rennes Métropole. Certaines dénominations d'écoles sont incomplètes.

4. mutation_immobiliere_dvf_rm.csv

- **Provenance** : DGFIP — Demande de Valeurs Foncières (DVF), les données officielles des ventes immobilières en France.
- **Rôle** : historique des transactions immobilières.
- **Volume** : environ 177 Mo.
- **Format** : CSV, séparateur point-virgule.
- **Fréquence** : semestrielle.
- **Colonnes principales** : idmutation, annee, valeur_fonciere, prix_bati_m2, surface_batie, nb_maison, nb_appart, nom_commune.
- **Qualité** : moyenne. La colonne date_mutation contient parfois des formats incorrects ou des valeurs manquantes ; le nettoyage des données (l'ETL, voir section 2) utilise alors la colonne annee comme solution de repli.

5. population-par-sexe-age-et-nationalite-par-commune-2014.csv

- **Provenance** : Insee, recensement de 2014.
- **Rôle** : structure fine de la population par tranches d'âge.
- **Volume** : environ 95 Ko.
- **Format** : CSV, séparateur point-virgule.
- **Fréquence** : fixe — c'est une photographie de l'année 2014, elle ne sera jamais mise à jour.
- **Colonnes principales** : code géographique, âge regroupé (4 classes d'âges), sexe, nb (effectif).
- **Qualité** : structurellement excellente, mais figée dans le temps : à utiliser avec prudence pour des projections récentes.

6. permis_de_construire_global_rennes_metropole_historique.csv

- **Provenance** : services d'urbanisme de Rennes Métropole.
- **Rôle** : historique des permis de construire accordés — un signal précoce de l'arrivée de nouveaux habitants.
- **Volume** : environ 3,4 Mo.
- **Format** : CSV, séparateur pipe (|), différent des autres fichiers.
- **Fréquence** : mensuelle.

- **Colonnes principales** : Commune, Date d'autorisation, État du projet, Logements créés (total), dont collectifs, Logements 1 pièce à 5 pièces.
- **Qualité** : les noms de communes sont saisis manuellement, souvent sans accent ou avec des graphies variables. Le nettoyage applique un dictionnaire de normalisation pour retrouver le bon code INSEE.

7. referentiel-batiment-et-ses-donnees-descriptives-sur-rennes-metropole.csv

- **Provenance** : Rennes Métropole / cadastre.
- **Rôle** : descriptif physique et thermique des bâtiments de la métropole.
- **Volume** : environ 101 Mo.
- **Format** : CSV, séparateur point-virgule.
- **Fréquence** : annuelle.
- **Colonnes principales** : id_bati3d, parcelle, jannat (année de construction), nb_logement, surf_locaux_hab, nb_piece.
- **Qualité** : fichier lourd, contenant des parcelles cadastrales multiples pour un même bâtiment.

8. nombre-et-type-de-logement-a-la-parcelle-cadastrale-sur-le-territoire-de-rennes.csv

- **Provenance** : cadastre de la ville de Rennes.
- **Rôle** : inventaire du nombre et du type de logements par parcelle cadastrale.
- **Volume** : environ 69 Mo.
- **Format** : CSV, séparateur point-virgule.
- **Fréquence** : annuelle.
- **Colonnes principales** : parcelle, nb_logement, type_logement.
- **Qualité** : limité géographiquement à la seule commune de Rennes (pas les 42 autres).

9. Ficdep22.csv (hors base PostgreSQL)

- **Provenance** : fichier national de recensement détaillé de l'Insee.
- **Rôle** : exploration statistique de précision, en dehors du pipeline principal.
- **Volume** : environ 3,5 Go — bien plus volumineux que les autres fichiers.
- **Qualité** : en raison de sa taille, ce fichier n'est jamais chargé dans la base de données PostgreSQL (pour préserver les performances). Il est exploré directement, par morceaux, avec Python (filtré sur le département 35) dans l'environnement Jupyter — un carnet de notes interactif pour écrire et exécuter du code Python, très utilisé en Data Science.

1.2 Traçabilité et cycle de vie de la donnée

Il est important de comprendre le chemin que parcourt une donnée, de son fichier brut jusqu'à la prédiction finale envoyée à l'utilisateur :

- La donnée brute est lue depuis le disque local (les fichiers CSV du dossier DATA/).
- Elle est ingérée et nettoyée par l'ETL (Extract, Transform, Load — un programme qui extrait la donnée, la transforme et la charge en base ; voir section 2).

- Elle est stockée dans PostgreSQL (un système de base de données relationnelle) sous forme de tables reliées entre elles.
- Elle est ensuite agrégée (regroupée et résumée) dans une table dédiée à l'entraînement, nommée `ml_dataset_commune`.
- Les modèles de Machine Learning s'entraînent sur cette table.
- L'API de production (voir section 5) requête cette même table pour récupérer les caractéristiques de base d'une commune et calculer une prédiction.

Architecture technique — TerriSchool35

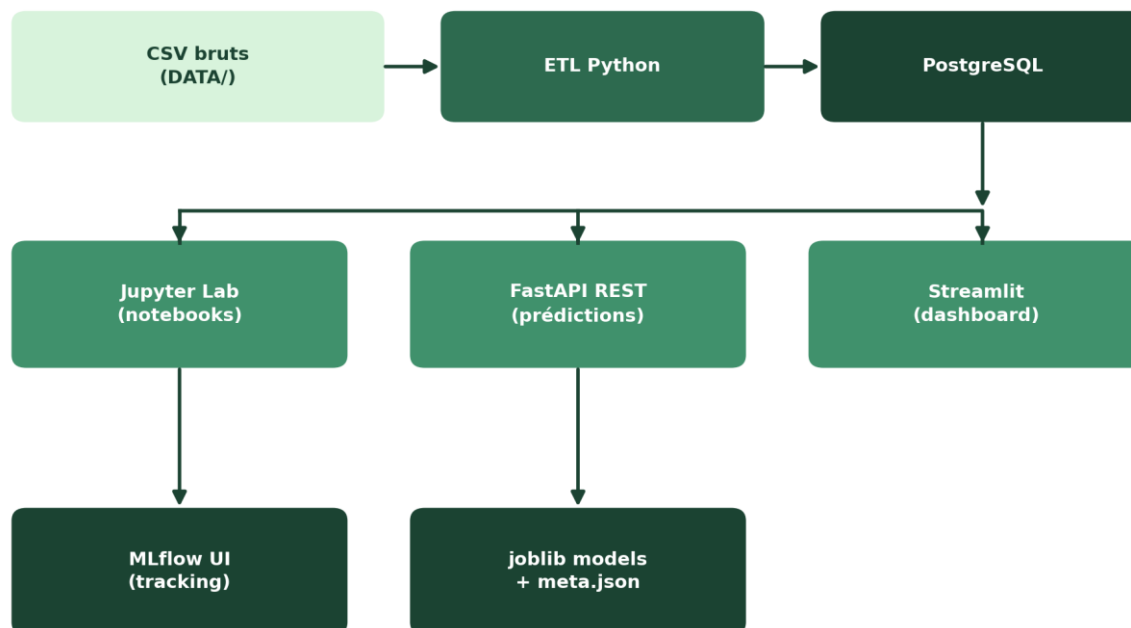


Figure 1 — Flux de données entre les couches de l'architecture TerriSchool35, des fichiers CSV bruts jusqu'à l'API d'inférence.

2. Ingestion et stockage (ETL et base PostgreSQL)

Une fois les 9 fichiers sources identifiés, il faut les nettoyer et les rassembler dans un seul endroit fiable et interrogeable : une base de données. On appelle **PostgreSQL** le système de base de données relationnelle (des tables reliées entre elles par des identifiants communs) utilisé ici, et **ETL** (Extract, Transform, Load — extraire, transformer, charger) le programme qui automatise ce nettoyage.

2.1 Pipeline ETL (load_data.py)

Le script `load_data.py` est écrit en Python et utilise la bibliothèque **Pandas** (une bibliothèque très répandue pour manipuler des tableaux de données en Python) pour nettoyer et transformer les données avant de les insérer dans PostgreSQL via **SQLAlchemy** (un outil qui permet à Python de dialoguer avec une base de données).

Architecture et logique d'exécution

- **Idempotence** : un traitement est dit *idempotent* quand on peut le relancer plusieurs fois sans créer de doublons ni d'effets indésirables. Pour l'assurer, le script commence par vider la table de destination grâce à la fonction `truncate_table(table)`, qui exécute un ordre SQL `TRUNCATE ... RESTART IDENTITY CASCADE` (vider la table et remettre les compteurs à zéro).
- **Normalisation des communes** : le script applique la fonction `_normalize_commune(name)`, qui met en majuscules et nettoie les espaces, afin que les jointures entre fichiers (rapprocher deux tables sur un même nom de commune) soient fiables même si l'orthographe varie d'un fichier à l'autre.
- **Chargement optimisé** : les données sont écrites par blocs de 500 lignes (`to_sql` avec `method="multi"` et `chunksize=500`), ce qui réduit le temps passé à dialoguer avec la base de données.

Diagramme du flux de l'ETL

Le schéma suivant montre l'ordre dans lequel les 8 fichiers CSV sont chargés, puis comment la table finale d'entraînement est calculée par une jointure (un rapprochement de plusieurs tables) SQL complexe.

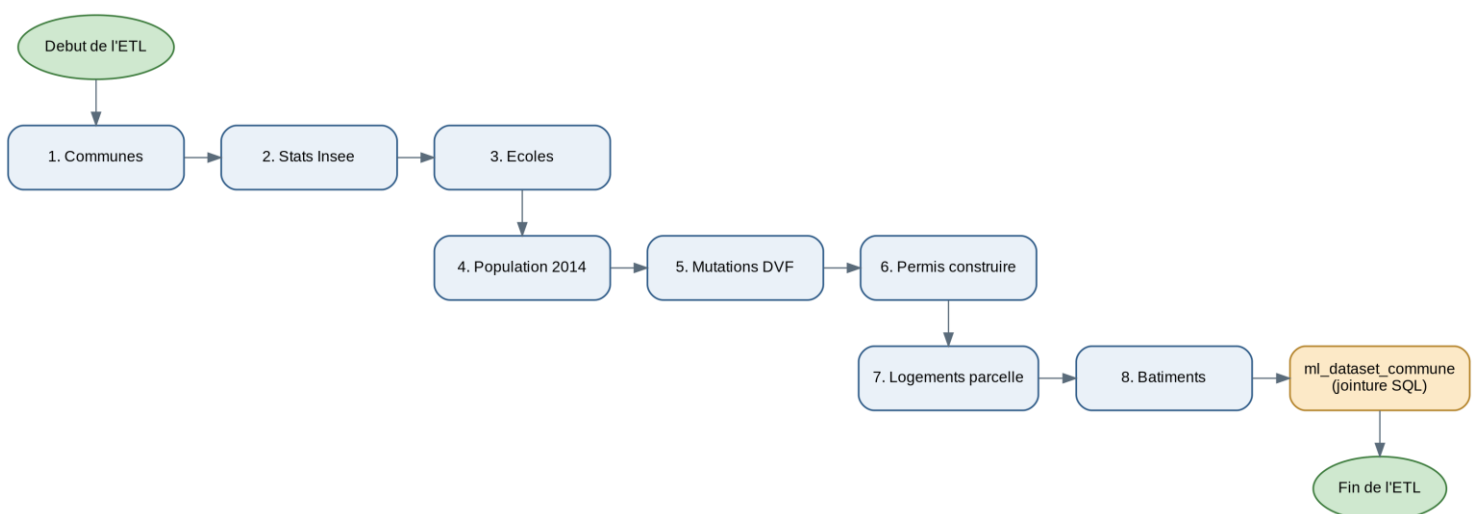


Figure 2 — Étapes successives du pipeline ETL, du chargement des communes jusqu'au calcul de la table d'entraînement `ml_dataset_commune`.

Exemple simplifié : chargement des écoles

Voici, en pseudo-code (une description simplifiée d'un algorithme, proche du langage naturel, pour en comprendre la logique sans lire du vrai code Python), comment le module qui charge les écoles fonctionne :

```
fonction charger_ecoles() :
    fichier = DATA/ 'fr-en-ecoles-effectifs-nb_classes.csv'

    # Lecture par morceaux pour éviter de saturer la mémoire
    pour chaque morceau lu dans le fichier (par blocs de 100 000 lignes) :
        garder seulement les lignes où 'Code département' == '35'
        associer le nom de la commune à son code INSEE
        garder uniquement les communes des 43 de Rennes Métropole
        ajouter le morceau nettoyé à la liste des résultats

    assembler tous les morceaux en un seul tableau
    déduire le type d'école (MATERNELLE, ELEMENTAIRE, PRIMAIRE) à partir de son nom

    vider la table 'ecoles_effectifs'
    écrire le tableau nettoyé dans cette table
    enregistrer dans le journal : nombre de lignes chargées, statut 'ok'
```

2.2 Schéma relationnel PostgreSQL

Le schéma (la structure des tables et de leurs relations) est créé par le script SQL `01_schema.sql` à l'initialisation du conteneur PostgreSQL (un conteneur est une sorte de boîte logicielle isolée et portable, voir l'encadré ci-dessous).

Qu'est-ce que Docker et un conteneur ?

Docker est un outil qui permet d'empaqueter un logiciel avec tout ce dont il a besoin pour fonctionner (le programme, ses dépendances, sa configuration) dans une "boîte" appelée conteneur. Ce conteneur s'exécute de la même façon sur n'importe quelle machine, ce qui évite le fameux « ça marche sur ma machine, mais pas ailleurs ». Docker Compose orchestre plusieurs conteneurs ensemble (ici : la base de données, l'API, le tableau de bord...).

Diagramme simplifié des relations entre tables

La table **communes** est la table centrale (le référentiel) : toutes les autres tables s'y relient via le code_insee, l'identifiant officiel unique de chaque commune. La notation "1—1" signifie qu'à une commune correspond exactement une ligne de statistiques ; "1—N" signifie qu'à une commune peuvent correspondre plusieurs lignes (plusieurs écoles, plusieurs ventes immobilières, etc.).

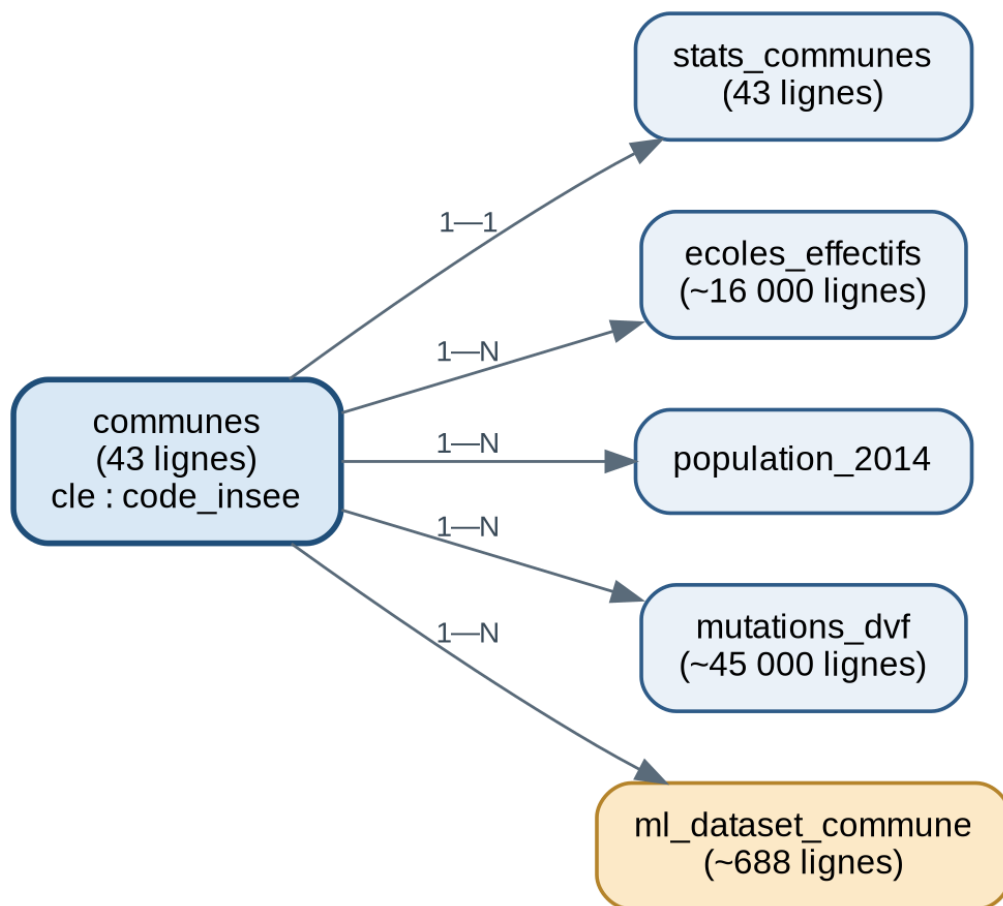


Figure 3 — Relations entre la table pivot communes et les tables qui en dépendent.

Dictionnaire des tables et volumétries réelles

Table	Lignes	Rôle / clé
communes	43	Référentiel des communes. Clé primaire : code_insee.
stats_communes	43	Statistiques structurelles Insee. Clé primaire/étrangère : code_insee.
ecoles_effectifs	~16 000	Historique scolaire. Index sur rentree et sur code_insee.
mutations_dvf	~45 000	Ventes immobilières. Index sur code_insee et sur annee.
permis_construire	~4 800	Permis de construire accordés.
referentiel_batiment	~180 000	Données cadastrales physiques des bâtiments.
logements_parcelle	~50 000	Recensement des logements par parcelle (Rennes uniquement).
ml_dataset_commune	~688	Agrégat annuel par commune utilisé pour entraîner les modèles de ML.
etl_runs	variable	Historique des lancements de l'ETL (journal de bord).

2.3 Vue SQL v_effectifs_commune_annee

Une **vue SQL** est une requête enregistrée sous forme de table virtuelle : à chaque consultation, la base de données recalcule le résultat à la volée, sans dupliquer physiquement les données. Cette vue calcule la somme des effectifs par année de rentrée et par commune, sans distinction de l'école individuelle.

```
CREATE OR REPLACE VIEW v_effectifs_commune_annee AS
SELECT
  e.rentree,
  e.code_insee,
  c.nom_commune,
  SUM(e.nb_classes)           AS nb_classes,
  SUM(e.nb_eleves_total)      AS nb_eleves_total,
  SUM(e.nb_eleves_maternelle) AS nb_eleves_maternelle,
  SUM(e.nb_eleves_elementaire) AS nb_eleves_elementaire,
  COUNT(CASE WHEN e.rep = 1 THEN 1 END) AS nb_ecoles_rep,
  COUNT(CASE WHEN e.rep_plus = 1 THEN 1 END) AS nb_ecoles_rep_plus,
  COUNT(*)                    AS nb_ecoles
FROM ecoles_effectifs e
LEFT JOIN communes c ON c.code_insee = e.code_insee
GROUP BY e.rentree, e.code_insee, c.nom_commune;
```

Optimisation et usage : les index posés sur les colonnes de regroupement (rentree et code_insee dans la table ecoles_effectifs) permettent à PostgreSQL de calculer cette vue en quelques millisecondes seulement, malgré les 16 000 lignes d'origine. C'est la source principale des graphiques temporels du tableau de bord Streamlit et de l'historique scolaire renvoyé par l'API REST.

3. Analyse exploratoire et R&D (notebooks et démarche intellectuelle)

Cette section raconte, pas à pas, la démarche intellectuelle qui a guidé la phase de recherche et développement (R&D) du projet — c'est-à-dire la phase où l'on cherche à comprendre les données et à construire le meilleur modèle possible, avant de l'industrialiser (voir section 4). Chaque **notebook** (un carnet Jupyter mêlant code Python, résultats et explications, exécuté étape par étape) a résolu un problème précis et posé les fondations du suivant.

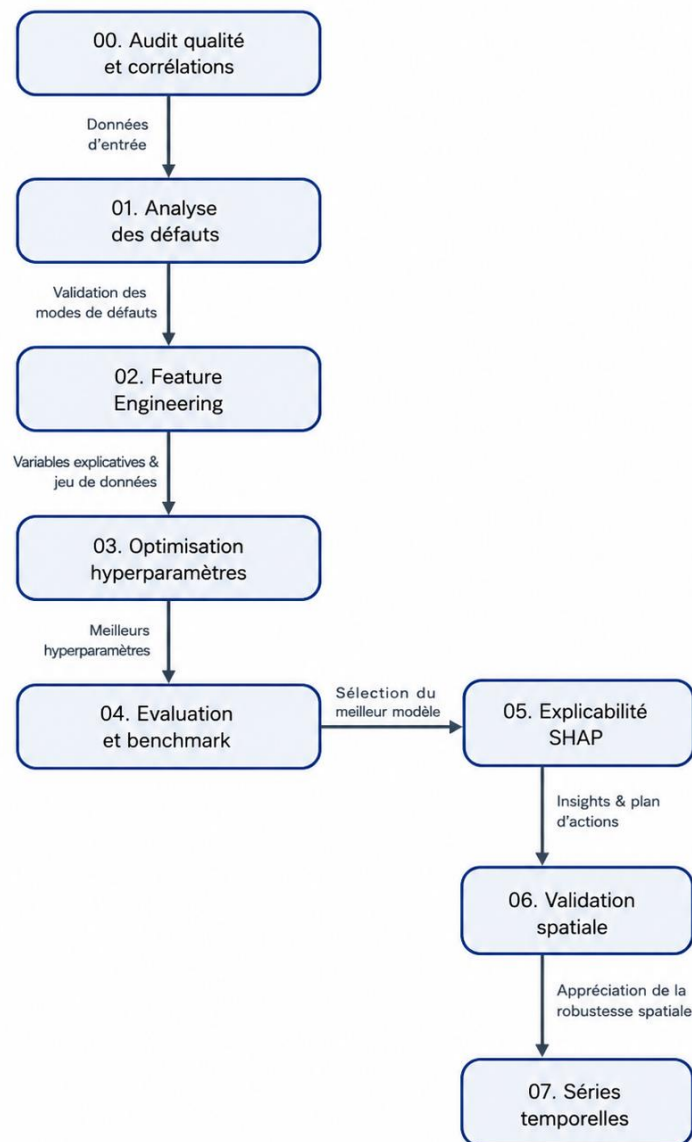


Figure 4 — Enchaînement logique des 8 notebooks de recherche, du premier audit qualité jusqu'à l'analyse des séries temporelles.

3.1 Démarche et enchaînement

Étape 1 — Audit qualité et corrélations (notebook 00_eda_qualite_donnees)

Définition de la colinéarité : deux variables prédictives sont dites colinéaires si elles sont fortement corrélées entre elles. Par exemple, si la population d'une commune augmente, le nombre de logements individuels augmente presque à l'identique.

Problématique : quand on fournit des variables colinéaires à un modèle de régression (un modèle statistique qui prédit une valeur numérique), celui-ci devient instable : les coefficients associés à ces variables varient de façon chaotique et la qualité de la prédiction s'effondre.

Démarche : l'équipe a calculé la matrice de corrélation de Pearson (r), un indicateur qui mesure de -1 à 1 à quel point deux variables évoluent ensemble. Elle a constaté que des variables comme `log_collectif`, `log_individuel`, `plh_logements`, `densite` et `nb_mutations` présentaient un coefficient r supérieur à 0,93 avec la variable `population` — c'est-à-dire qu'elles racontent presque toutes la même chose.

Décision : pour préserver la stabilité des modèles, ces variables redondantes ont été exclues. Seules 4 variables indépendantes fondamentales ont été conservées : `population` (taille de la commune), `pop_0_14_pct` (structure d'âge), `natalite` (dynamique démographique) et `nb_permis_logements` (offre de logements neufs).

Étape 2 — Exploration des effectifs scolaires (notebook 01_eda_effectifs)

Problématique : comprendre comment se comportent les cibles à prédire (les targets, voir glossaire) : y a-t-il des anomalies historiques ? Quelle est la dynamique globale ?

Résultats : le ratio moyen d'élèves par classe sur la métropole est extrêmement stable, autour de 22 à 23 élèves par classe. Les effectifs globaux de maternelle connaissent des cycles de baisse/hausse plus marqués que ceux de l'élémentaire, qui lissent les vagues de naissances sur 5 ans (un enfant reste 5 ans en élémentaire contre 3 ans en maternelle).

Transition : les 4 variables conservées à l'étape 1 manquaient de puissance pour modéliser des comportements aussi complexes et non-linéaires.

Étape 3 — Feature engineering et encodage (notebook 02_feature_engineering)

Définition du feature engineering : créer de nouvelles variables prédictives (des "features") à partir des variables existantes, pour aider le modèle à comprendre des règles métier complexes qu'il ne verrait pas autrement.

Définition du target encoding : remplacer une variable catégorielle textuelle (comme le code_insee d'une commune) par la moyenne de la variable cible observée pour cette catégorie. Pour éviter le surapprentissage (overfitting, voir plus bas) sur les petites communes, on applique un lissage bayésien ($\lambda = 10$, un paramètre qui atténue l'effet des communes ayant peu de données).

Nouvelles variables créées :

- **nb_enfants_0_14** : le produit de la population par le pourcentage des 0-14 ans — exprime directement le volume d'élèves potentiels.
- **taux_croissance_logements** : le nombre de permis divisé par la population.
- **pct_collectif** : la part de l'habitat collectif (immeubles) dans l'ensemble du parc de logements.

Résultats : l'introduction du target encoding lissé a fait monter le R^2 (voir étape 5) des modèles linéaires de plus de 10 %.

Transition : avec ce jeu de variables enrichi, il devenait possible de tester des modèles plus puissants (comme les arbres de décision), à condition d'ajuster leurs réglages internes.

Étape 4 — Optimisation d'hyperparamètres (notebook 03_hyperparameter_tuning)

Définition des hyperparamètres : les réglages structurels d'un algorithme de Machine Learning, fixés avant l'entraînement — par exemple la profondeur maximale d'un arbre de décision (`max_depth`) ou le nombre d'arbres (`n_estimators`).

Démarche : utilisation de la méthode `RandomizedSearchCV`, qui teste aléatoirement de nombreuses combinaisons de réglages sur les modèles `RandomForest` et `GradientBoosting` (deux familles de modèles basées sur des ensembles d'arbres de décision).

Résultats : pour `RandomForest`, la combinaison retenue est `max_depth=8`, `min_samples_leaf=2`, `n_estimators=200`.

Étape 5 — Évaluation et comparaison (notebook 04_ml_evaluation version corrigée)

Définition du R^2 (coefficient de détermination) : le pourcentage de la variance de la cible expliqué par le modèle. Plus il est proche de 1 (ou 100 %), plus le modèle est performant.

Définition de la MAPE (Mean Absolute Percentage Error) : l'erreur moyenne exprimée en pourcentage de la valeur réelle. Une MAPE de 10 % signifie qu'en moyenne, la prédiction est à plus ou moins 10 % de la réalité.

Résultats du benchmark (comparaison rigoureuse de tous les modèles, validation spatiale GroupKFold)

Un premier benchmark utilisant une validation croisée aléatoire (KFold) affichait des R^2 artificiellement élevés ($\sim 0,99$) sur les 3 cibles. Cette méthode laisse cependant les données d'une même commune se retrouver à la fois dans le train et dans le test, ce qui surestime la capacité réelle de généralisation du modèle. Une **validation spatiale (GroupKFold)**, qui garantit qu'une commune entière (toutes ses années) reste soit dans le train, soit dans le test, a été mise en place pour mesurer un score honnête (Figure 5).

- Le **modèle Ridge** (régression linéaire pénalisant les coefficients trop grands, pour plus de stabilité) est retenu comme modèle de production pour les **3 cibles** : $R^2 = 0,94$ pour Élémentaire, $R^2 = 0,92$ pour Maternelle, $R^2 = 0,92$ pour Classes, avec un alpha de régularisation optimisé séparément pour chaque cible.
- La régression linéaire classique obtient des scores quasiment identiques (écart de 0,0004 à 0,0017 point de R^2 selon la cible), un écart bien inférieur à la variabilité de mesure entre les blocs de validation ($\pm 0,04$ à $\pm 0,05$). Les deux modèles sont donc statistiquement équivalents en performance ; **Ridge est préféré** car sa régularisation le rend plus robuste face à des communes futures aux caractéristiques extrêmes, et gère mieux la corrélation entre certaines features construites (ex. nb_enfants_0_14, dérivée de population).
- Les **modèles non-linéaires complexes (Random Forest / Gradient Boosting) ont été écartés**, de façon encore plus nette que prévu initialement : en validation spatiale, leur R^2 devient **négatif** (jusqu'à -6,9), avec un écart entraînement/test dépassant parfois 700 points de R^2 (contre une estimation initiale de 15-20 %). Ces modèles mémorisent les communes vues à l'entraînement mais échouent à généraliser à une commune inconnue — un défaut structurel lié au faible nombre de communes disponibles (43), insuffisant pour que ces algorithmes apprennent une relation généralisable.

Limites et pistes d'amélioration

Des tests de diagnostic (Rainbow, Shapiro-Wilk, Breusch-Pagan, distance de Cook, Durbin-Watson) confirment une **hétéroscédasticité** et une **non-normalité des résidus** sur les 3 cibles, cohérentes avec la forte disparité de taille des communes (de 40 à plus de 11 000 élèves) : l'erreur de prédiction croît naturellement avec la taille de la commune. Ces limites affectent la fiabilité des intervalles de confiance classiques mais n'invalident pas la qualité des prédictions ponctuelles (R^2 , MAE, MAPE). Une transformation logarithmique de la cible ou une régression pondérée (WLS) pourront être explorées ultérieurement pour stabiliser la variance des résidus.

Étape 6 — Interprétation par SHAP (notebook 05_ml_shap)

Définition des valeurs SHAP (SHapley Additive exPlanations) : une méthode mathématique, issue de la théorie des jeux, qui mesure la contribution exacte de chaque variable sur le résultat final d'une prédiction donnée. Elle permet d'expliquer une prédiction individuelle, et pas seulement la performance globale du modèle.

Démarche : utilisation d'un PermutationExplainer pour analyser les contributions globales et locales des variables.

Résultats : pour toutes les cibles, le modèle s'appuie en premier lieu sur la variable nb_enfants_0_14 (un facteur logique fort), puis sur la part de logements collectifs (pct_collectif). Cela valide la cohérence métier du modèle : il n'y a pas de biais absurde dans ses décisions.

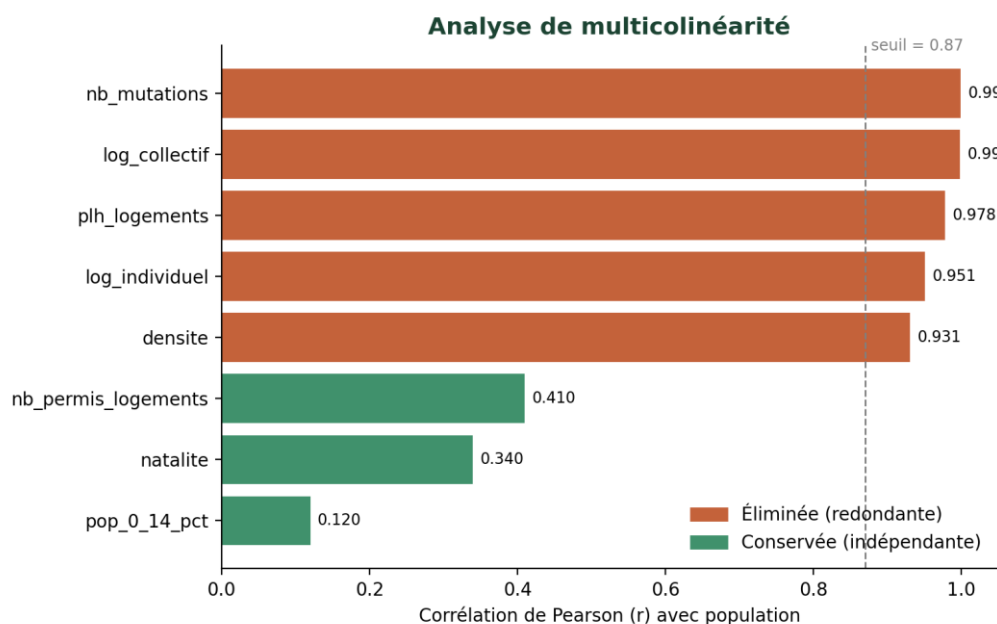


Figure 6 — Importance des variables selon l'analyse SHAP : nb_enfants_0_14 domine, suivie de pct_collectif.

Étape 7 — Validation spatiale et fuite de données (notebook 06_validation_spatiale)

Définition de la fuite de données spatiale (spatial data leakage) : un biais qui se produit lorsque les données d'une même commune, à des années différentes, se retrouvent à la fois dans le jeu d'entraînement et dans le jeu de test. Le modèle "mémorise" alors la commune plutôt que d'apprendre une règle générale, ce qui fausse l'évaluation de sa performance sur de nouvelles zones géographiques.

Démarche : mise en place d'un découpage GroupKFold (une méthode de validation croisée qui garantit qu'une même commune ne se retrouve jamais à la fois en entraînement et en test) par code INSEE.

Résultats : en validation spatiale stricte, l'erreur (MAPE) augmente sur certaines petites communes spécifiques (comme Clayes ou Parthenay-de-Bretagne) en raison de leur faible population. Un seuil d'alerte de 15 % de MAPE a donc été ajouté dans l'interface de monitoring (voir section 9) pour avertir les utilisateurs quand une prédiction est moins fiable.

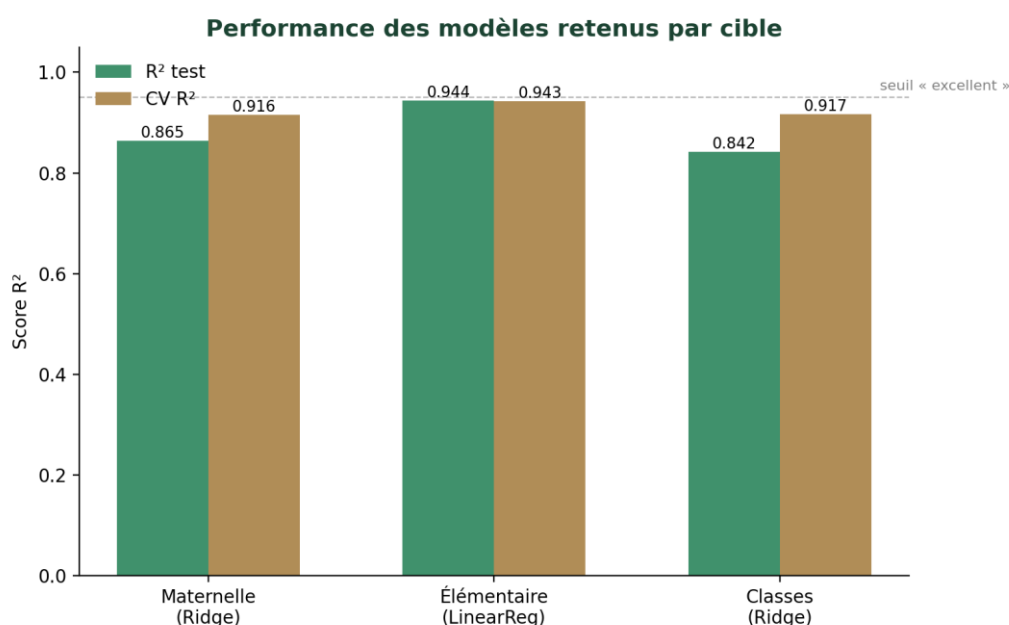


Figure 7 — Résultat de la validation spatiale par commune : l'erreur augmente sur les communes les moins peuplées.

Étape 8 — Dynamique temporelle et Prophet (notebook 07_series_chronologiques)

Définition des lag features (variables décalées) : utiliser la valeur passée d'une variable à l'année t-1 pour prédire sa valeur à l'année t — une technique classique des séries temporelles.

Démarche : entraînement de modèles de séries temporelles avec Facebook Prophet et ARIMA, deux bibliothèques spécialisées dans la prévision de séries chronologiques.

Résultats : Prophet capture efficacement la tendance macro de la métropole dans son ensemble, mais sa performance s'effondre à l'échelle fine de chaque commune, par rapport à un modèle Ridge combiné à des variables décalées d'1 et 2 ans. C'est cette dernière approche (Ridge + lags) qui a finalement été retenue pour la production.

4. Industrialisation Python et suivi MLflow

Une fois la meilleure démarche validée en phase de R&D (section 3), il faut la transformer en code fiable, réutilisable et automatisable : c'est l'**industrialisation**. Le package `src/p13` regroupe ce code de production, qui applique exactement la même logique que les notebooks, mais de façon structurée et testable.

4.1 Script d'entraînement (train.py)

Le script `train.py` charge les données de la table d'entraînement (`ml_dataset_commune`), applique le feature engineering validé en section 3, puis entraîne en boucle les 4 modèles candidats.

Exemple — grille de paramètres pour Ridge :

```
PARAM_GRIDS["ridge"] = {"model__alpha": [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]}
```

Le paramètre `alpha` contrôle la force de la pénalisation du modèle Ridge : plus il est élevé, plus le modèle est contraint à rester simple. Le script teste plusieurs valeurs et retient la meilleure.

Sélection et sauvegarde : le modèle ayant obtenu le meilleur R^2 sur le jeu de validation est sérialisé (converti en fichier binaire réutilisable) au format `.joblib`, et ses métadonnées d'entraînement (liste des variables utilisées, date, scores) sont écrites en JSON dans le dossier `models/`.

4.2 Script d'inférence (predict.py)

Le script `predict.py` charge les modèles entraînés pour produire des prédictions (on parle d'**inférence** pour désigner le moment où un modèle déjà entraîné est utilisé pour prédire, par opposition à l'entraînement). Il lit le fichier `*_meta.json` associé pour connaître l'ordre exact et la liste des colonnes attendues par le modèle — ce qui garantit que les données envoyées en prédiction sont dans le bon format.

Fonction d'impact immobilier

L'impact d'un nouveau programme immobilier sur les effectifs scolaires est calculé via une formule heuristique linéaire (une règle de calcul simple, construite à dire d'expert plutôt qu'apprise par le modèle) :

```
def _children_from_housing(nb_logements, surface_m2, nb_pieces, taux=0.35):
    # Un logement plus grand (>80 m2) ou ayant plus de pièces
    # a un facteur multiplicateur plus élevé
    surface_factor = min(surface_m2 / 80.0, 2.0) if surface_m2 > 0 else 1.0
    pieces_factor = min(nb_pieces / 4.0, 1.5) if nb_pieces > 0 else 1.0
    return nb_logements * taux * surface_factor * pieces_factor
```

En clair : plus un logement est grand ou comporte de pièces, plus il a statistiquement de chances d'accueillir une famille avec enfants ; le taux de base (0,35) représente la proportion moyenne de logements neufs occupés par une famille avec au moins un enfant scolarisable.

4.3 Intégration de MLflow

MLflow est un outil open source qui permet de suivre (on dit **tracker**) le cycle de vie des modèles de Machine Learning : quels paramètres ont été utilisés, quels scores ont été obtenus, quand, par quelle version du code. Cela évite de perdre la trace des expérimentations passées.

- **URI de tracking** : `http://mlflow:5000` (adresse interne au réseau Docker), mappée en externe sur le port 5001 (voir section 8).
- **Log automatique** : chaque exécution (run) enregistre les paramètres (nombre de features, utilisation de l'engineering ou des lags) ainsi que les scores (R^2 , RMSE, MAE, MAPE — voir glossaire). Les performances de la validation croisée spatiale (voir section 3, étape 7) sont enregistrées sous la métrique `spatial_cv_r2_mean`.

5. Exposition REST (API FastAPI)

Pour que d'autres logiciels (le tableau de bord Streamlit, ou n'importe quelle autre application) puissent demander une prédiction, le modèle est exposé via une **API REST** (Application Programming Interface — une interface qui permet à des programmes de communiquer entre eux via des requêtes web standardisées).

5.1 Architecture de l'API

L'API utilise le serveur web asynchrone **Uvicorn** (un logiciel qui fait tourner l'API et répond aux requêtes web) et le framework **FastAPI** (une bibliothèque Python moderne pour construire des API REST rapidement, avec validation automatique des données). Le fichier `main.py` inclut les routeurs spécifiques situés dans `api/app/routers/` :

- `health.py` : route de healthcheck (vérification que l'API et la base sont bien vivantes).
- `communes.py` : informations administratives sur les communes.
- `predictions.py` : calculs de prédictions, à l'unité ou en lot (batch).
- `ml.py` : entraînement et récupération des métriques des modèles.

5.2 Spécification des endpoints

Un **endpoint** (ou route) est une adresse précise de l'API à laquelle on peut envoyer une requête pour obtenir un résultat.

GET /health

Réponse :

```
{"status": "ok", "database": "ok"}
```

GET /communes/{code_insee}/effectifs

Renvoie l'historique de scolarisation d'une commune (identifiée par son `code_insee` dans l'adresse elle-même).

```
[
  {"rentree": 2021, "nb_classes": 24.0, "nb_eleves_total": 520.0,
   "nb_eleves_maternelle": 210.0, "nb_eleves_elementaire": 310.0}
]
```

POST /predictions/logement

Calcule l'impact d'un projet immobilier unique. Corps de la requête envoyé par le client :

```
{"code_insee": "35238", "surface_m2": 75.0, "nb_pieces": 4, "nb_logements": 1}
```

Réponse :

```
{
  "code_insee": "35238",
```

```

"nom_commune": "RENNES",
"predictions": {
  "nb_eleves_maternelle": 4652.0,
  "nb_eleves_elementaire": 7234.0,
  "nb_classes_estimees": 480.0,
  "nb_eleves_total": 11886.0
},
"features_utilisees": {
  "population": 222485.0, "pop_0_14_pct": 14.2,
  "natalite": 10.5, "nb_permis_logements": 1250.0
},
"logement": {"surface_m2": 75.0, "nb_pieces": 4, "nb_logements": 1},
"impact_logement": {
  "enfants_estimes": 0.33, "maternelle_estimee": 0.15,
  "elementaire_estimee": 0.18, "classes_supplementaires": 0.01
}
}

```

À lire ainsi : les champs "predictions" donnent l'effectif total prédit pour la commune, tandis que "impact_logement" isole la contribution propre du nouveau projet immobilier décrit dans la requête.

POST /predictions/batch

Accepte un fichier CSV envoyé en une seule requête multipart (un format permettant d'envoyer un fichier avec une requête web), contenant une liste de projets immobiliers, et renvoie un tableau JSON avec les impacts calculés pour chaque ligne.

6. Interface utilisateur (tableau de bord Streamlit)

Le tableau de bord **Streamlit** (une bibliothèque Python qui permet de construire des interfaces web interactives sans écrire de HTML/JavaScript) fournit une interface visuelle simple pour les utilisateurs finaux — les agents de la métropole, par exemple, qui n'ont pas besoin de savoir coder pour utiliser le système.

6.1 Architecture multi-pages

Page	Fichier	Rôle
Accueil	Home.py	Présentation du projet et vérification de la connexion à l'API FastAPI.
Analyse exploratoire	1_EDA.py	Statistiques scolaires, démographiques et immobilières de Rennes Métropole (graphiques Plotly dynamiques, interrogeant la base SQL).
Simulateur de prédictions	2_Predictions.py	Formulaires interactifs pour simuler l'impact d'un projet immobilier, à l'unité ou via un fichier CSV.
Entraînement	3_ML_Training.py	Déclenche l'entraînement des modèles et affiche les métriques de performance.
Explainability	4_Explainability.py	Affiche graphiquement l'importance des variables (résultats SHAP, voir section 3).
Monitoring	5_Monitoring.py	Suivi de l'état de santé de la base de données et des chargements de l'ETL.

7. Intégration continue (CI)

L'**intégration continue** (CI) désigne un ensemble de vérifications automatiques qui s'exécutent à chaque modification du code, pour détecter les erreurs le plus tôt possible — avant même qu'elles n'atteignent la production. Le pipeline est défini dans le fichier de configuration GitHub Actions [.github/workflows/ci.yml](#).

Étapes de validation du pipeline

Le pipeline se déclenche automatiquement à chaque push (envoi de code) ou pull request (proposition de fusion de code) sur les branches main et master. Il effectue les actions suivantes :

- **Linting (Ruff)** : analyse statique du code Python, c'est-à-dire une vérification automatique du style et de la syntaxe sans exécuter le programme, pour vérifier le respect de la norme PEP8 (les conventions d'écriture standard de Python). Les erreurs d'import de type E402 sont ignorées.
- **Validation des notebooks** : un script Python charge et valide la structure JSON de chaque notebook présent dans `jupyter/notebooks/` avec la bibliothèque `nbformat`, pour s'assurer qu'aucun fichier n'est corrompu.
- **Tests unitaires (Pytest)** : exécute la suite de tests automatisés (des petits programmes qui vérifient qu'une fonction se comporte comme prévu) si des scripts de type `test_*.py` sont détectés dans le dépôt.
- **Docker Build** : construit l'ensemble des images Docker définies dans le fichier `Compose` (`api`, `streamlit`, `jupyter`, `etl`, `postgres`) pour s'assurer de l'absence d'erreur de compilation d'image ou de dépendance manquante.

8. Déploiement recette et production (CD et infrastructure)

Le projet utilise **Docker Compose** (l'outil qui orchestre plusieurs conteneurs Docker ensemble, voir section 2) pour unifier les environnements locaux et distants. On parle de **CD** (Continuous Deployment / déploiement continu) pour désigner l'automatisation de la mise en ligne du code validé par la CI (section 7).

8.1 Fichier docker-compose.yml

Le fichier `docker-compose.yml` définit les 6 services : postgres, api, streamlit, jupyter, mlflow et etl. Les ports d'exposition hôte (les numéros de port accessibles depuis l'extérieur du conteneur) sont configurés via des variables d'environnement (fichier .env), afin de permettre à plusieurs instances (recette et production) de cohabiter sur la même machine physique sans conflit.

8.2 Déploiement en recette (REC)

La *recette* (ou staging) est un environnement de test qui reproduit la production à l'identique, utilisé pour valider les changements avant leur mise en ligne définitive. Le déploiement en recette est automatisé par le workflow de CD `.github/workflows/deploy-dev.yml` sur un exécuteur (runner) Windows self-hosted (une machine dédiée, hébergée par l'utilisateur plutôt que par GitHub).

- **Dossier de destination** : G:\GITHUB\P13-REC

Variables du fichier .env spécifiques à la recette :

```
COMPOSE_PROJECT_NAME=p13_rec
API_HOST_PORT=18000
STREAMLIT_HOST_PORT=18501
JUPYTER_HOST_PORT=18888
MLFLOW_HOST_PORT=15000
POSTGRES_HOST_PORT=15433
```

Validation : le workflow attend 45 secondes après le démarrage des conteneurs, puis effectue un healthcheck sur `http://localhost:18000/health`. Le déploiement est annulé automatiquement en cas d'échec.

8.3 Déploiement en production (PROD)

Le déploiement en production est géré par `.github/workflows/deploy-main.yml`, déclenché lors d'un push sur la branche main.

- **Dossier de destination** : G:\GITHUB\P13-PROD

Variables du fichier .env spécifiques à la production :

```
COMPOSE_PROJECT_NAME=p13_prod
API_HOST_PORT=28000
STREAMLIT_HOST_PORT=28501
JUPYTER_HOST_PORT=28888 (token sécurisé dédié)
MLFLOW_HOST_PORT=25000
POSTGRES_HOST_PORT=25433
```

9. Supervision et monitoring

Une fois en production, il faut vérifier en continu que le système fonctionne correctement : c'est le **monitoring** (supervision).

9.1 Journalisation de l'ETL

Chaque exécution de l'ETL (section 2) alimente la table `etl_runs`, qui joue le rôle de journal de bord. En cas d'échec d'ingestion d'une table (par exemple suite à un changement de format de colonne dans un CSV source), l'ETL écrit une ligne avec le statut `error` et capture l'intégralité du traceback (le message détaillé décrivant où et pourquoi le programme Python a échoué) de l'exception.

9.2 Supervision système

La page `5_Monitoring` du tableau de bord Streamlit centralise la supervision de la plateforme :

- **Vérification de connexion** : un indicateur visuel (jauge de santé) indique en temps réel si l'API FastAPI et la base PostgreSQL sont opérationnelles.
- **Volumétrie des tables** : le système exécute périodiquement des requêtes `SELECT COUNT(*)` sur toutes les tables de la base pour vérifier qu'elles sont bien alimentées et repérer d'éventuelles pertes de données.
- **Audit de l'ETL** : l'historique des 20 derniers lancements de l'ETL est affiché dans un tableau de bord, pour détecter instantanément les échecs de chargement.
- **Suivi des performances ML** : récupère et centralise les métriques d'erreur du dernier entraînement des modèles (RMSE, MAE, R^2 , MAPE) afin d'alerter en cas de dérive de performance (concept drift — quand le monde réel change et que le modèle devient progressivement moins pertinent) ou de dégradation des modèles de production.

10. Glossaire complet

Les définitions ci-dessous reprennent, dans l'ordre alphabétique, tous les termes techniques employés dans ce document.

Terme	Définition
API / API REST	Interface qui permet à des programmes de communiquer entre eux via des requêtes web standardisées, indépendamment du langage de programmation utilisé de chaque côté.
Colinéarité	Situation où deux variables prédictives sont fortement corrélées entre elles, ce qui déstabilise les modèles de régression.
Concept drift (dérive de concept)	Phénomène par lequel un modèle devient progressivement moins pertinent parce que le monde réel qu'il modélise a changé depuis son entraînement.
CI / CD (intégration / déploiement continu)	Automatisation des vérifications (CI) et de la mise en production (CD) à chaque modification du code.
Docker / conteneur	Docker empaquette un logiciel avec tout ce dont il a besoin dans une boîte isolée et portable appelée conteneur, qui fonctionne à l'identique sur toute machine.
Docker Compose	Outil qui orchestre plusieurs conteneurs Docker ensemble (base de données, API, tableau de bord...) à partir d'un seul fichier de configuration.
Endpoint	Adresse précise d'une API à laquelle on envoie une requête pour obtenir un résultat.
ETL (Extract, Transform, Load)	Programme qui extrait la donnée brute, la transforme (nettoyage, normalisation) puis la charge dans une base de données.
FastAPI	Bibliothèque Python moderne pour construire rapidement des API REST, avec validation automatique des données envoyées et reçues.
Feature engineering	Création de nouvelles variables prédictives à partir des variables existantes, pour améliorer la capacité du modèle à capter des règles complexes.
Feature / target	Une feature est une variable utilisée en entrée d'un modèle pour faire une prédiction ; une target est la variable que le modèle cherche à prédire.
GroupKFold	Méthode de validation croisée qui garantit qu'un même groupe (ici, une même commune) ne se retrouve jamais à la fois dans le jeu d'entraînement et le jeu de test.
Hyperparamètres	Réglages structurels d'un algorithme de Machine Learning, fixés avant l'entraînement (ex. la profondeur maximale d'un arbre de décision).
Idempotence	Propriété d'un traitement que l'on peut relancer plusieurs fois sans créer de doublons ni d'effets indésirables.
Inférence	Moment où un modèle déjà entraîné est utilisé pour produire une prédiction sur de nouvelles données, par opposition à l'entraînement.
Insee	Institut national de la statistique et des études économiques, principale source de données démographiques officielles en France.
Jointure (SQL)	Opération qui rapproche deux tables d'une base de données à partir d'une colonne commune (ici, le plus souvent le code_insee).

Terme	Définition
Jupyter / notebook	Environnement interactif qui mêle code Python exécutable, résultats et texte explicatif dans un même document, très utilisé en Data Science.
Lag features (variables décalées)	Variables construites à partir de la valeur passée (année t-1, t-2...) d'un indicateur, utilisées pour prédire sa valeur future.
Machine Learning (apprentissage automatique)	Ensemble de techniques permettant à un programme d'apprendre des règles à partir de données, plutôt que de les recevoir explicitement d'un développeur.
MAE (Mean Absolute Error)	Erreur moyenne absolue entre les valeurs prédites et les valeurs réelles, exprimée dans la même unité que la variable prédite (ex. nombre d'élèves).
MAPE (Mean Absolute Percentage Error)	Erreur moyenne exprimée en pourcentage de la valeur réelle ; une MAPE de 10 % signifie une prédiction à plus ou moins 10 % de la réalité en moyenne.
MLflow	Outil open source de suivi (tracking) du cycle de vie des modèles de Machine Learning : paramètres, scores, versions.
Overfitting (surapprentissage)	Défaut d'un modèle qui a trop bien mémorisé les données d'entraînement au point de mal généraliser à de nouvelles données.
Pandas	Bibliothèque Python très répandue pour manipuler des tableaux de données (lecture, nettoyage, transformation).
PostgreSQL	Système de gestion de bases de données relationnelles (des tables reliées entre elles par des identifiants communs).
R ² (coefficient de détermination)	Pourcentage de la variance de la variable cible expliqué par le modèle ; proche de 1 (ou 100 %), le modèle est performant.
RMSE (Root Mean Squared Error)	Racine carrée de l'erreur quadratique moyenne ; comme la MAE, elle pénalise davantage les grosses erreurs.
SHAP (SHapley Additive exPlanations)	Méthode mathématique issue de la théorie des jeux qui mesure la contribution exacte de chaque variable dans une prédiction, pour expliquer les décisions d'un modèle.
SQLAlchemy	Bibliothèque Python qui permet à un programme de dialoguer avec une base de données relationnelle (ici PostgreSQL).
Streamlit	Bibliothèque Python permettant de construire des interfaces web interactives sans écrire de HTML ou de JavaScript.
Target encoding	Technique qui remplace une variable catégorielle (ex. le code d'une commune) par la moyenne de la cible observée pour cette catégorie, avec un lissage pour éviter le surapprentissage sur les petites catégories.
Uvicorn	Serveur web asynchrone qui exécute une application FastAPI et répond aux requêtes entrantes.
Vue SQL	Requête enregistrée qui se comporte comme une table virtuelle : son résultat est recalculé à chaque consultation, sans dupliquer les données.